



US012399636B2

(12) **United States Patent**
Vogelsang et al.

(10) **Patent No.:** **US 12,399,636 B2**

(45) **Date of Patent:** **Aug. 26, 2025**

(54) **MULTI-MODAL REFRESH OF DYNAMIC,
RANDOM-ACCESS MEMORY**

(56) **References Cited**

U.S. PATENT DOCUMENTS

(71) Applicant: **Rambus Inc.**, San Jose, CA (US)

(72) Inventors: **Thomas Vogelsang**, Jericho, VT (US);
Steven C. Woo, Saratoga, CA (US);
Michael Raymond Miller, Raleigh, NC
(US)

(73) Assignee: **Rambus Inc.**, San Jose, CA (US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 0 days.

5,487,172 A	1/1996	Hyatt
6,292,426 B1	9/2001	Ikeda et al.
7,565,480 B2	7/2009	Ware et al.
7,885,134 B2	2/2011	Li et al.
8,176,241 B2	5/2012	Hearn
8,797,822 B2	8/2014	Sato
2007/0195613 A1	8/2007	Rajan et al.
2011/0131371 A1	6/2011	Sun et al.
2013/0132704 A1	5/2013	Ware
2018/0158809 A1*	6/2018	Kim
2021/0241810 A1	8/2021	Hollis et al.

OTHER PUBLICATIONS

(21) Appl. No.: **18/655,510**

(22) Filed: **May 6, 2024**

(65) **Prior Publication Data**

US 2024/0354014 A1 Oct. 24, 2024

Related U.S. Application Data

(63) Continuation of application No. 17/503,058, filed on
Oct. 15, 2021, now Pat. No. 12,001,697.

(60) Provisional application No. 63/109,743, filed on Nov.
4, 2020.

(51) **Int. Cl.**
G06F 3/00 (2006.01)
G06F 3/06 (2006.01)

(52) **U.S. Cl.**
CPC **G06F 3/0634** (2013.01); **G06F 3/061**
(2013.01); **G06F 3/0673** (2013.01)

(58) **Field of Classification Search**
None
See application file for complete search history.

Bhati, Ishwar et al., "DRAM Refresh Mechanisms, Penalties, and
Trade-Offs", IEEE Transactions on Computers, vol. 64, No. X,
2015. 14 pages.

(Continued)

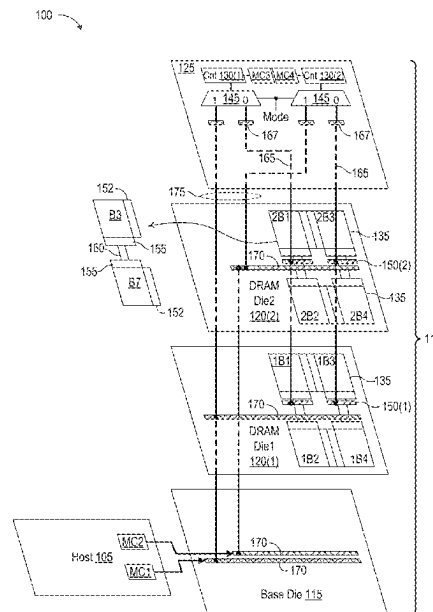
Primary Examiner — Midys Rojas

(74) *Attorney, Agent, or Firm* — Silicon Edge Law Group
LLP; Arthur J. Behiel

(57) **ABSTRACT**

A memory system includes two or more memory controllers
capable of accessing the same dynamic, random-access
memory (DRAM), one controller having access to the
DRAM or a subset of the DRAM at a time. Different subsets
of the DRAM are supported with different refresh-control
circuitry, including respective refresh-address counters.
Whichever controller has access to a given subset of the
DRAM issues refresh requests to the corresponding refresh-
address counter. Counters are synchronized before control of
a given subset of the DRAM is transferred between con-
trollers to avoid a loss of stored data.

20 Claims, 13 Drawing Sheets



(56)

References Cited

OTHER PUBLICATIONS

Chang, Kevin Kai-Wei et al., "Improving DRAM Performance by Parallelizing Refreshes with Accesses", Carnegie Mellon University 2014. 12 pages.

Elpida, "Low Power Function of Mobile RAM: Partial Array Self Refresh (PASR)," Technical Note, Document No. E0597E10 (Ver. 1.0), Published Nov. 2005. 5 pages.

* cited by examiner

100 

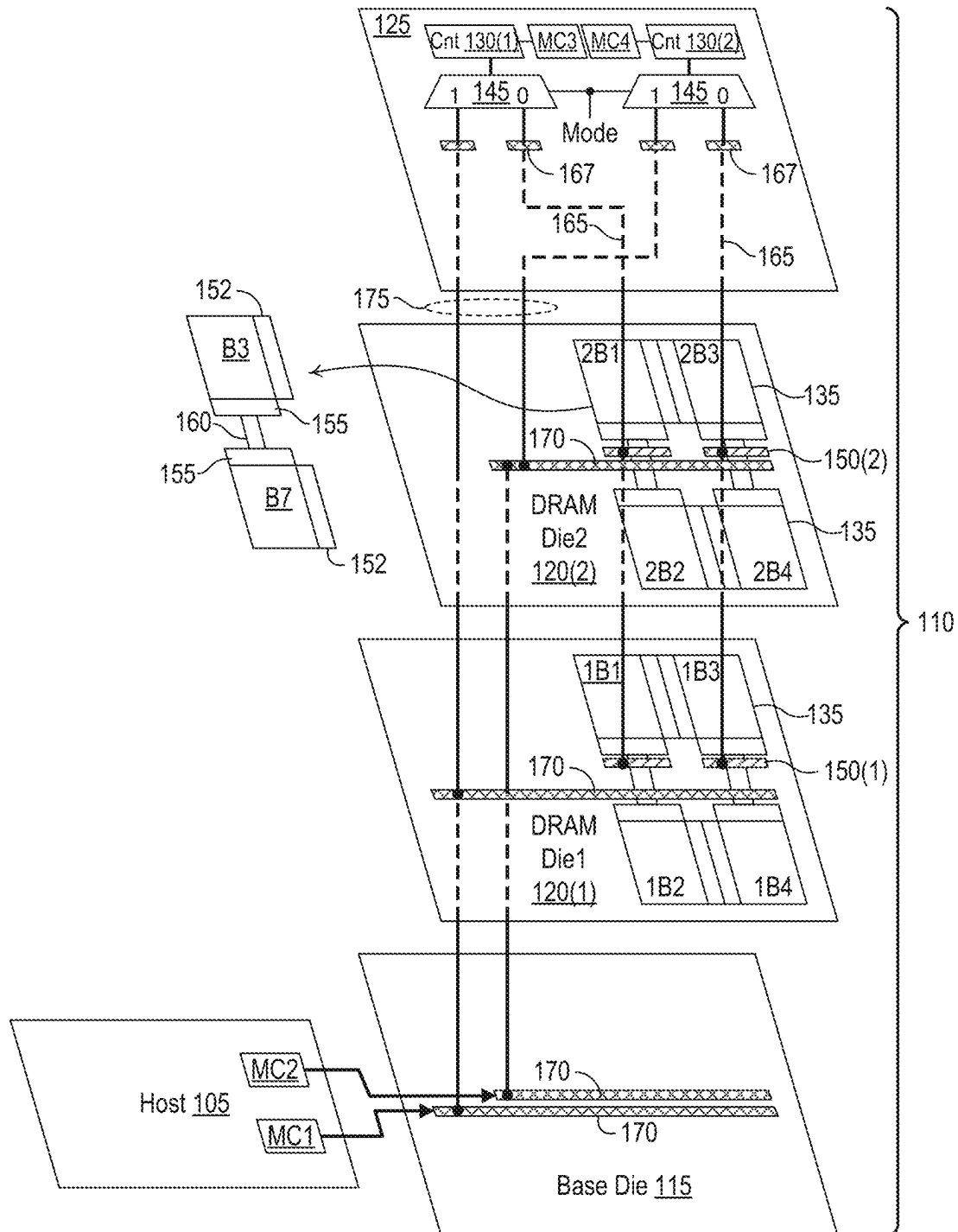


FIG. 1

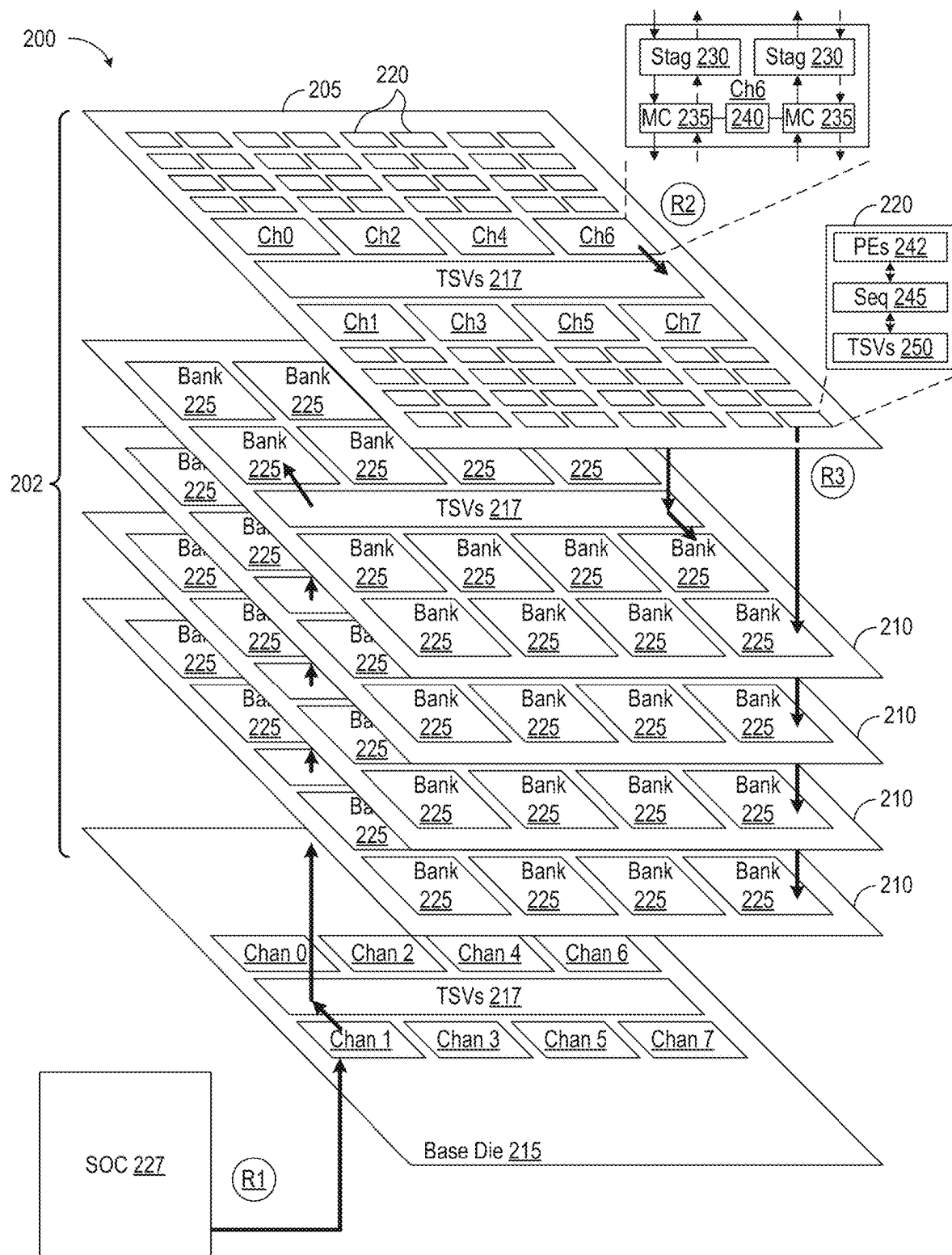
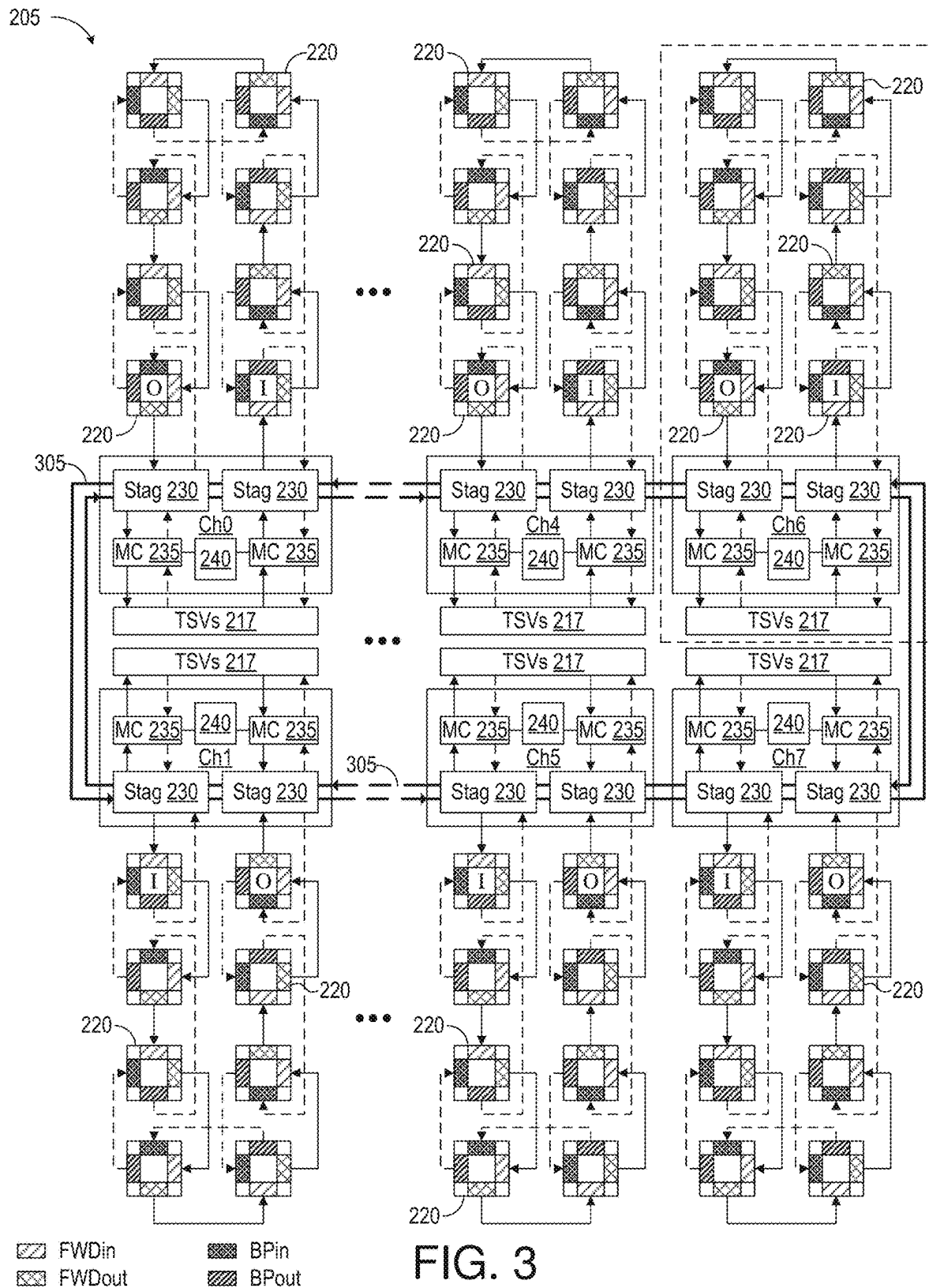


FIG. 2



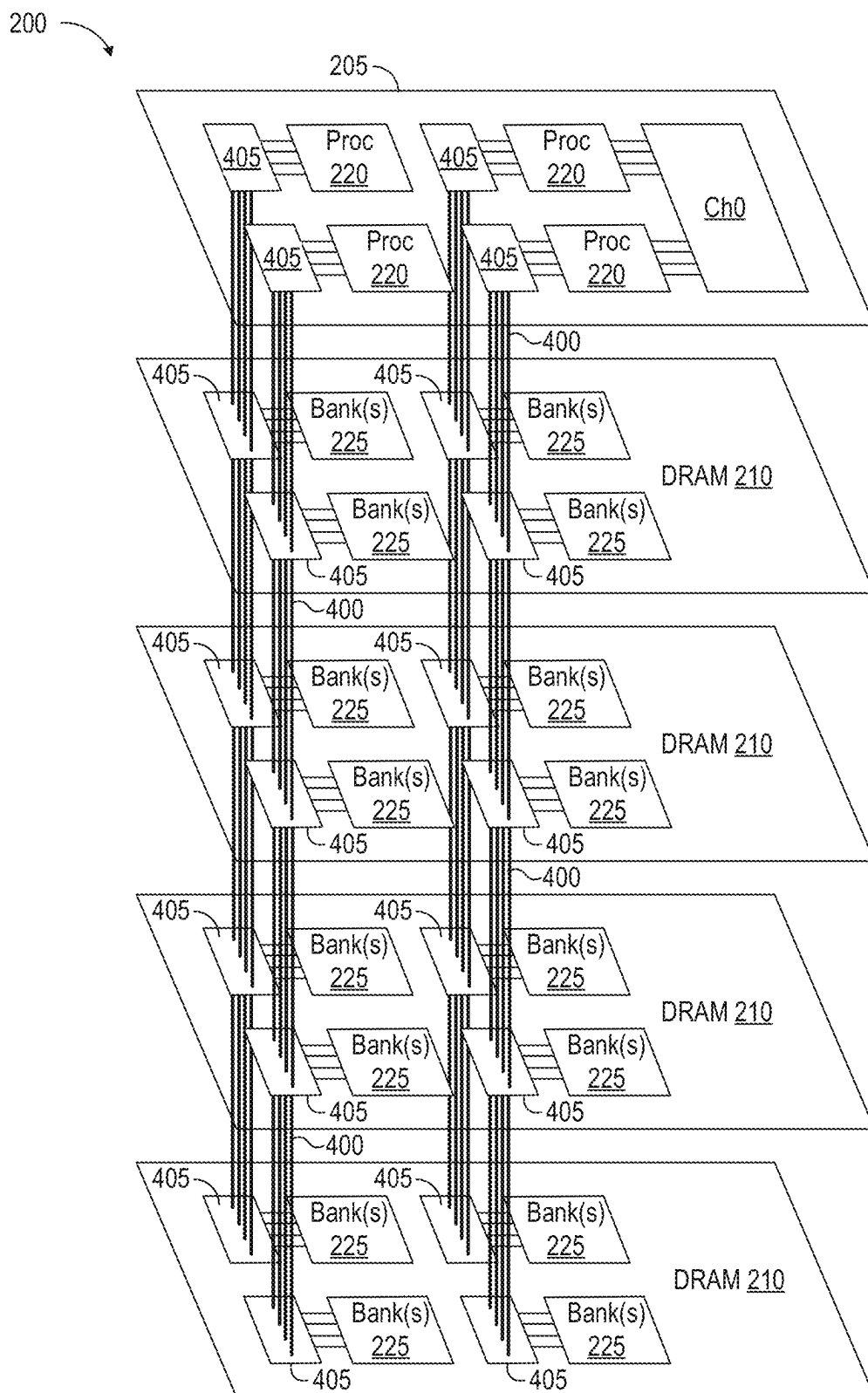


FIG. 4

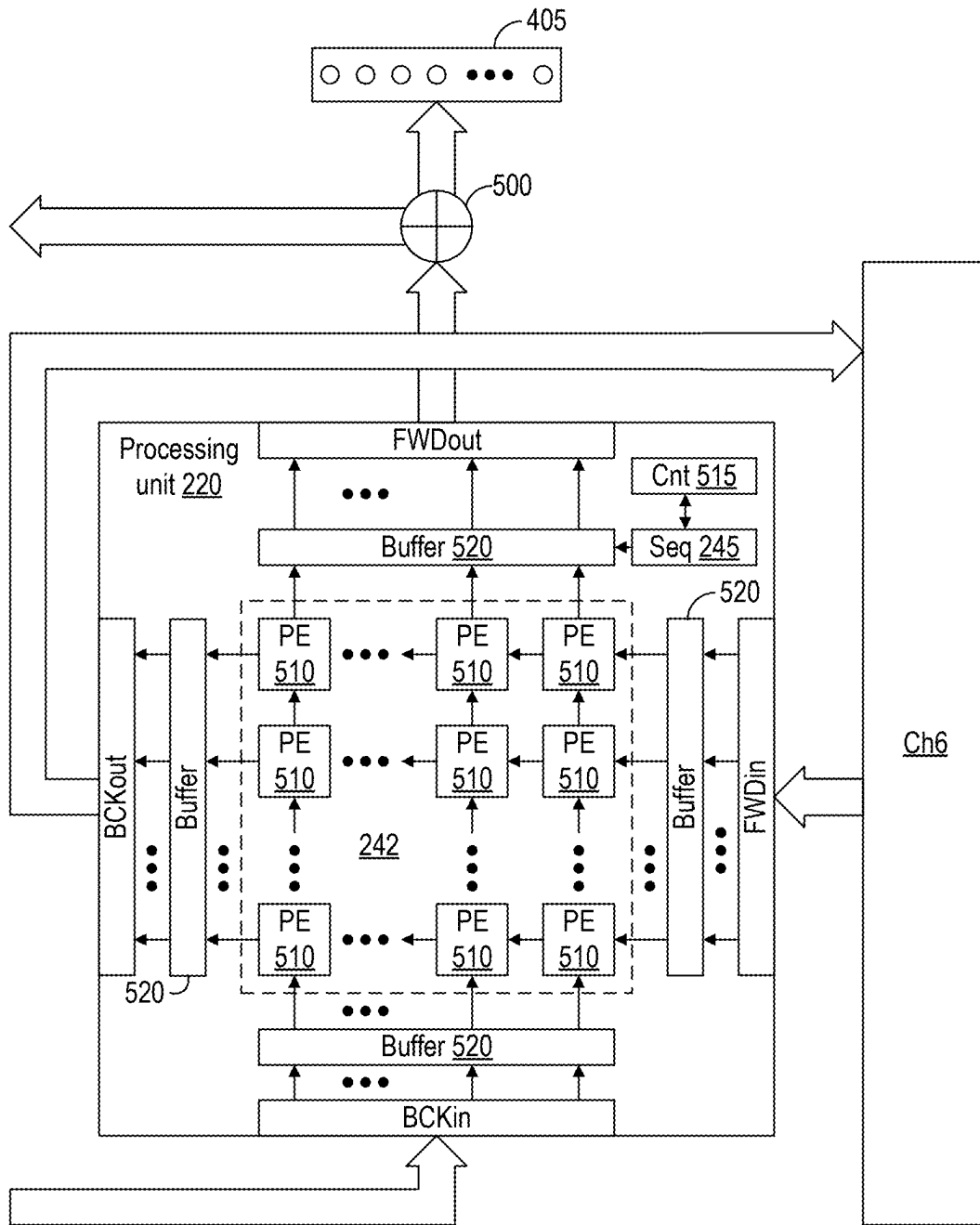


FIG. 5

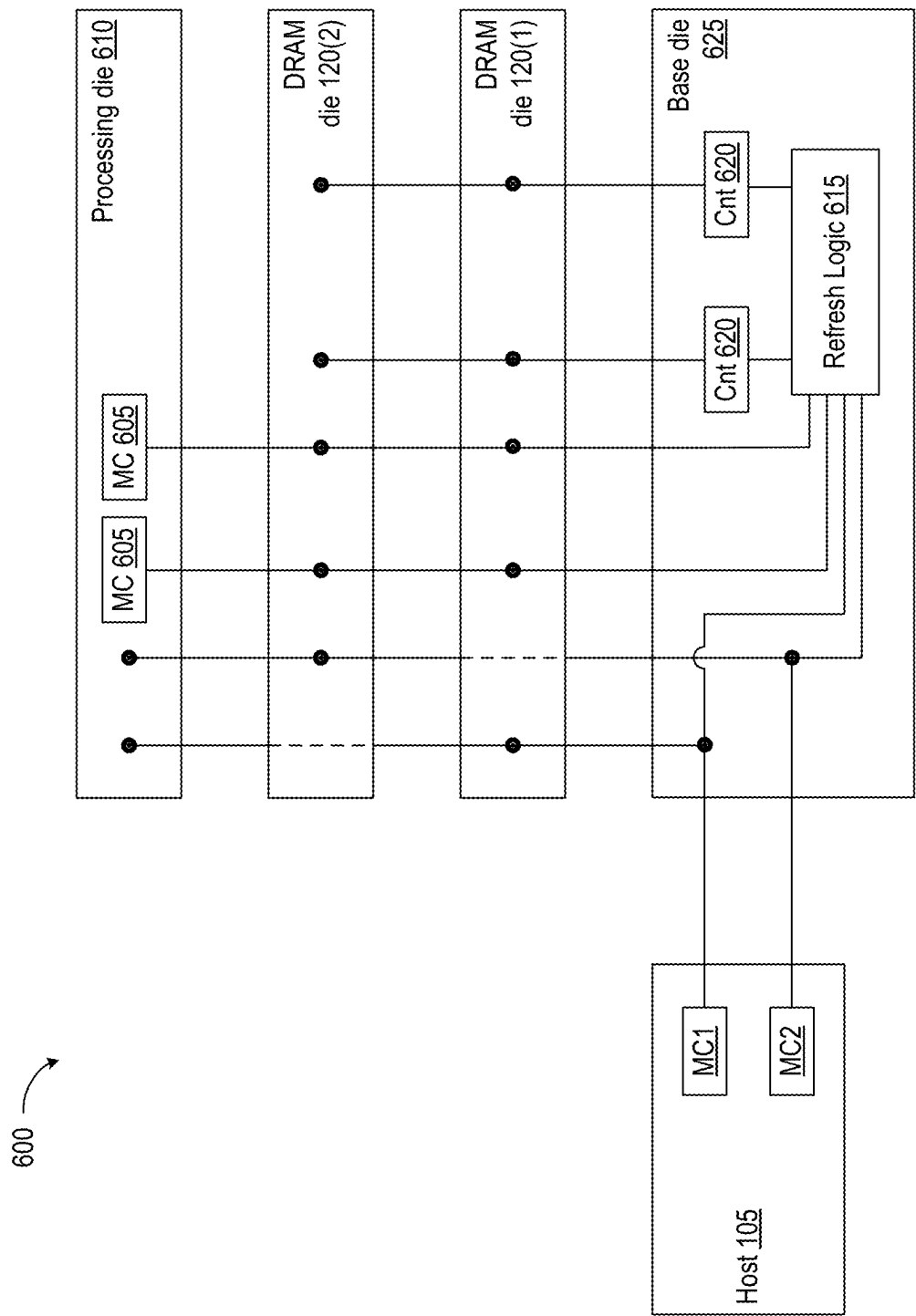


FIG. 6

700

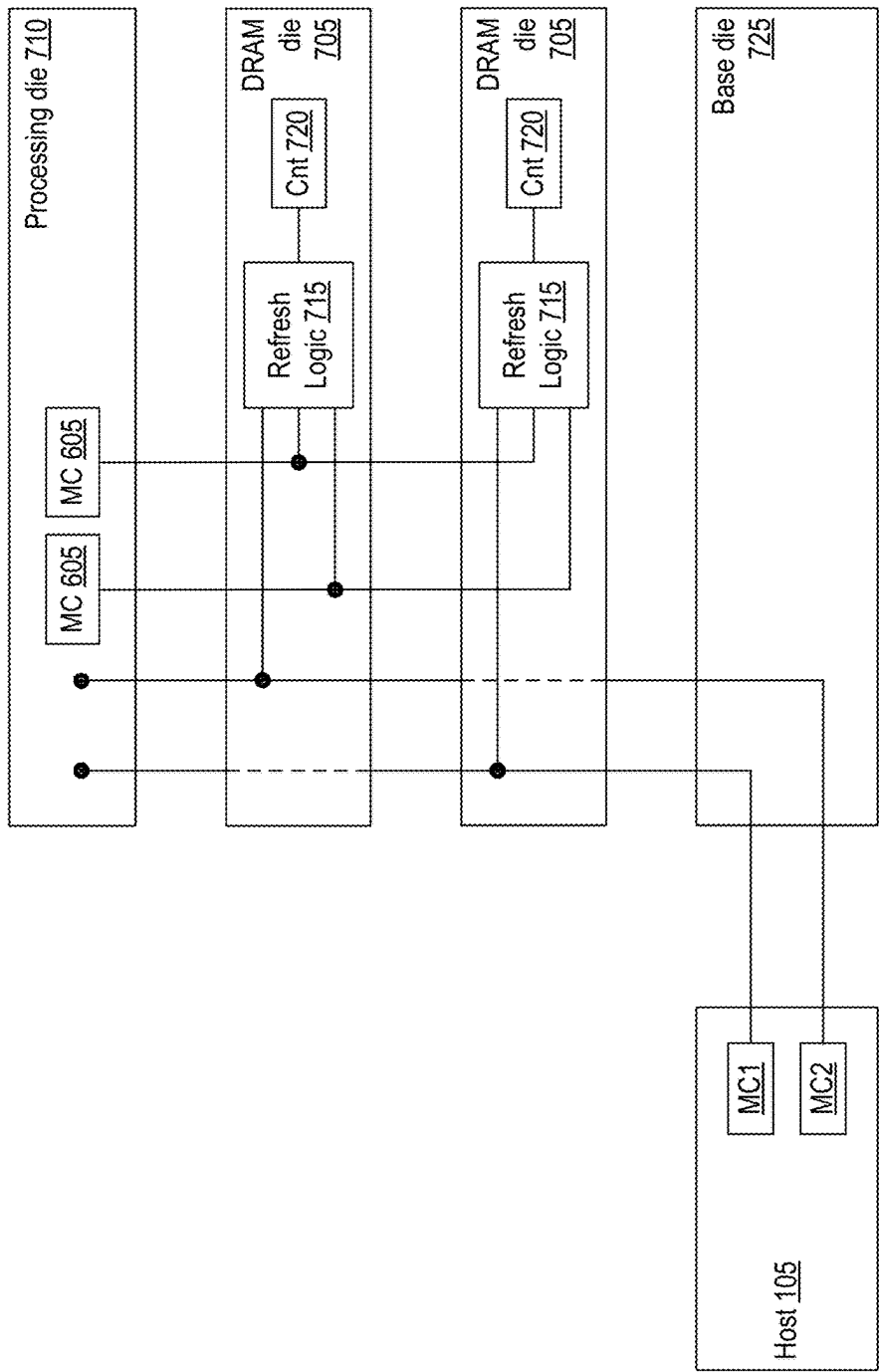
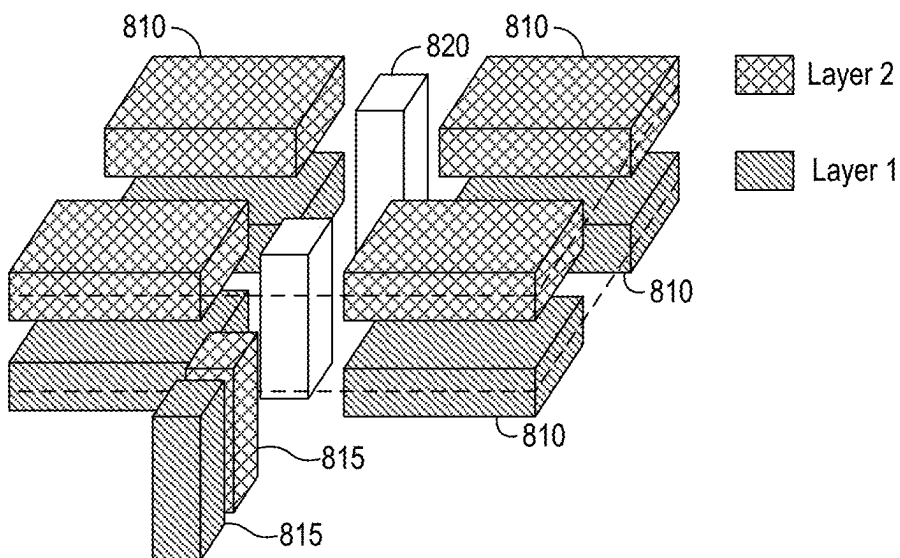


FIG. 7

800



805

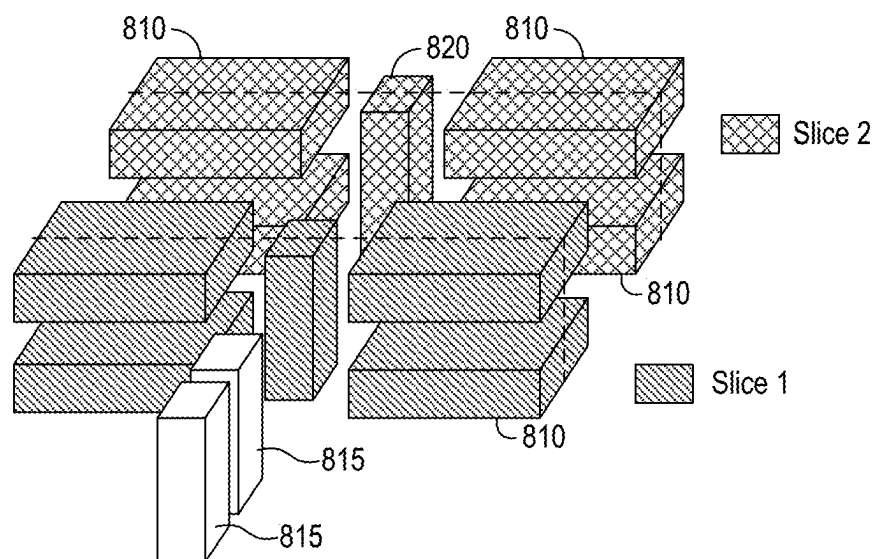


FIG. 8

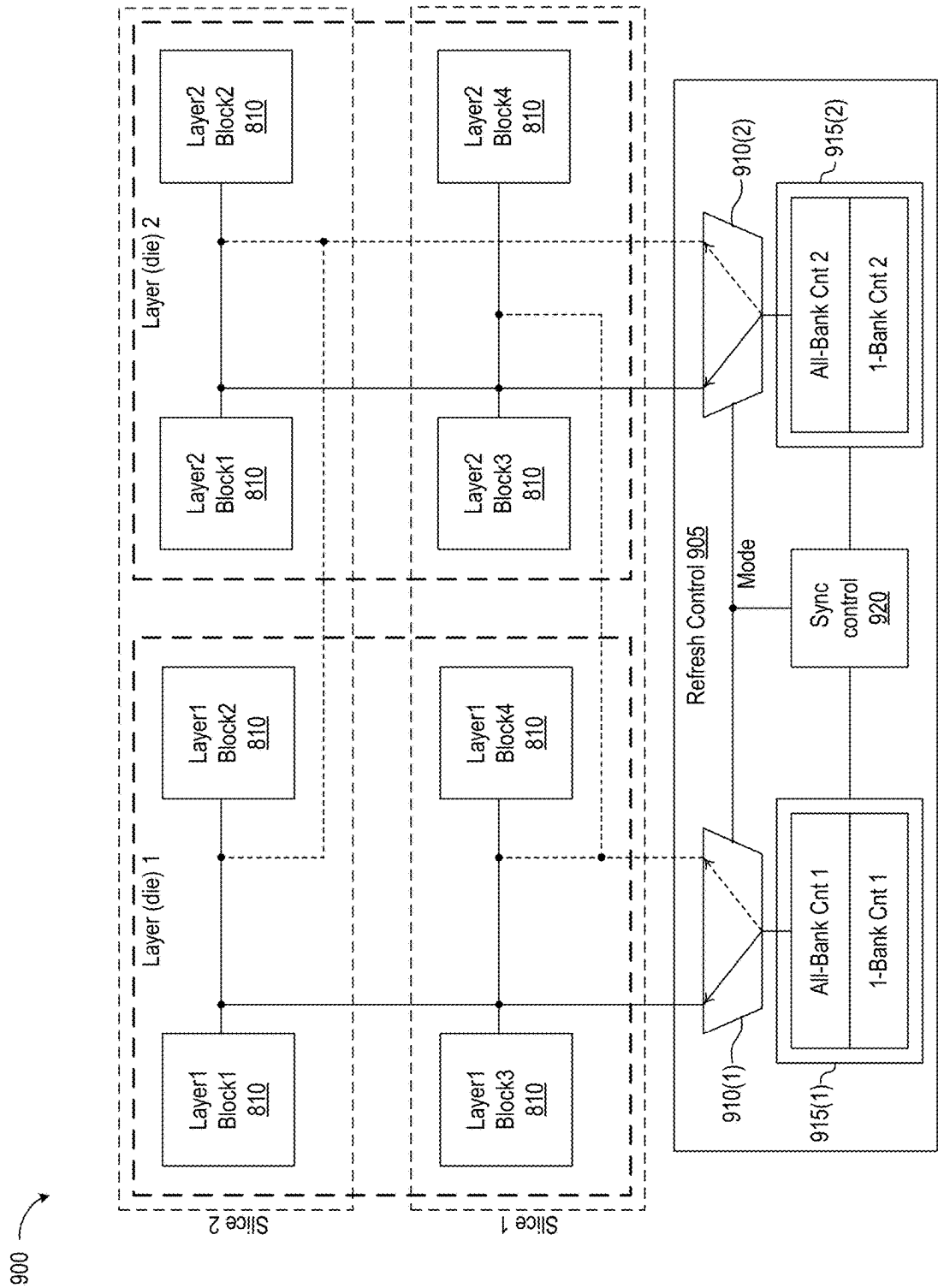


FIG. 9

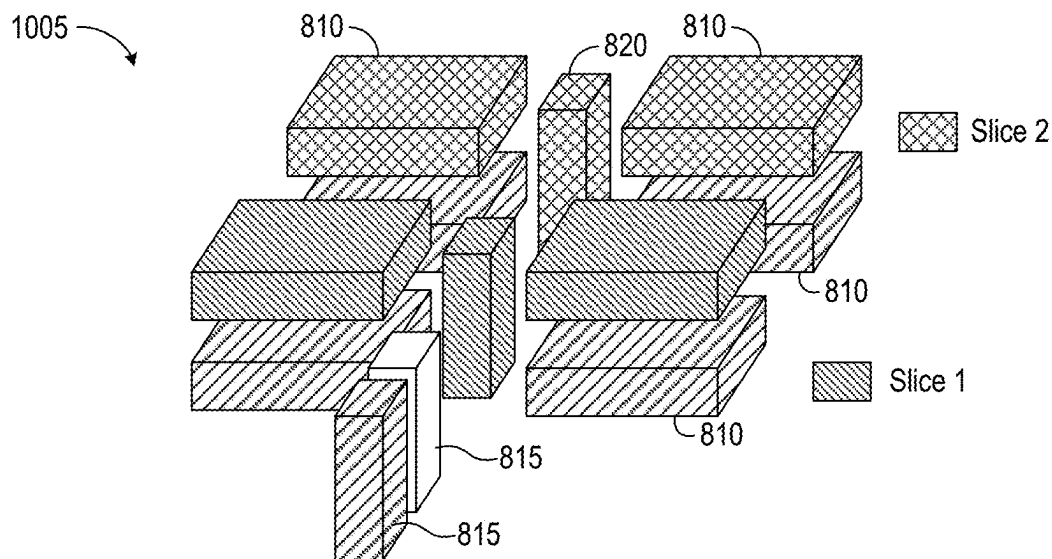
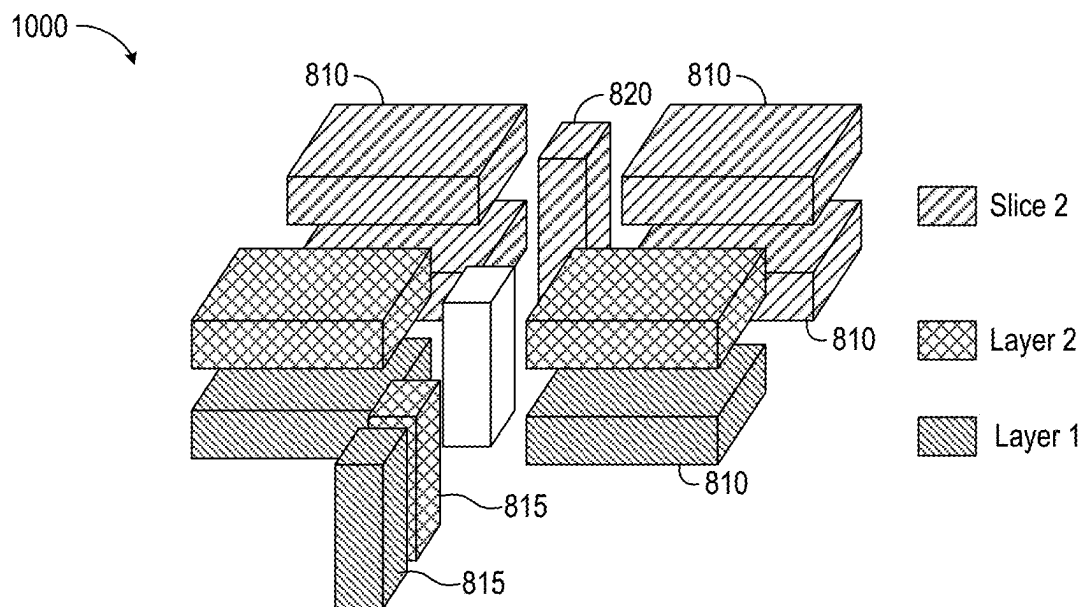


FIG. 10

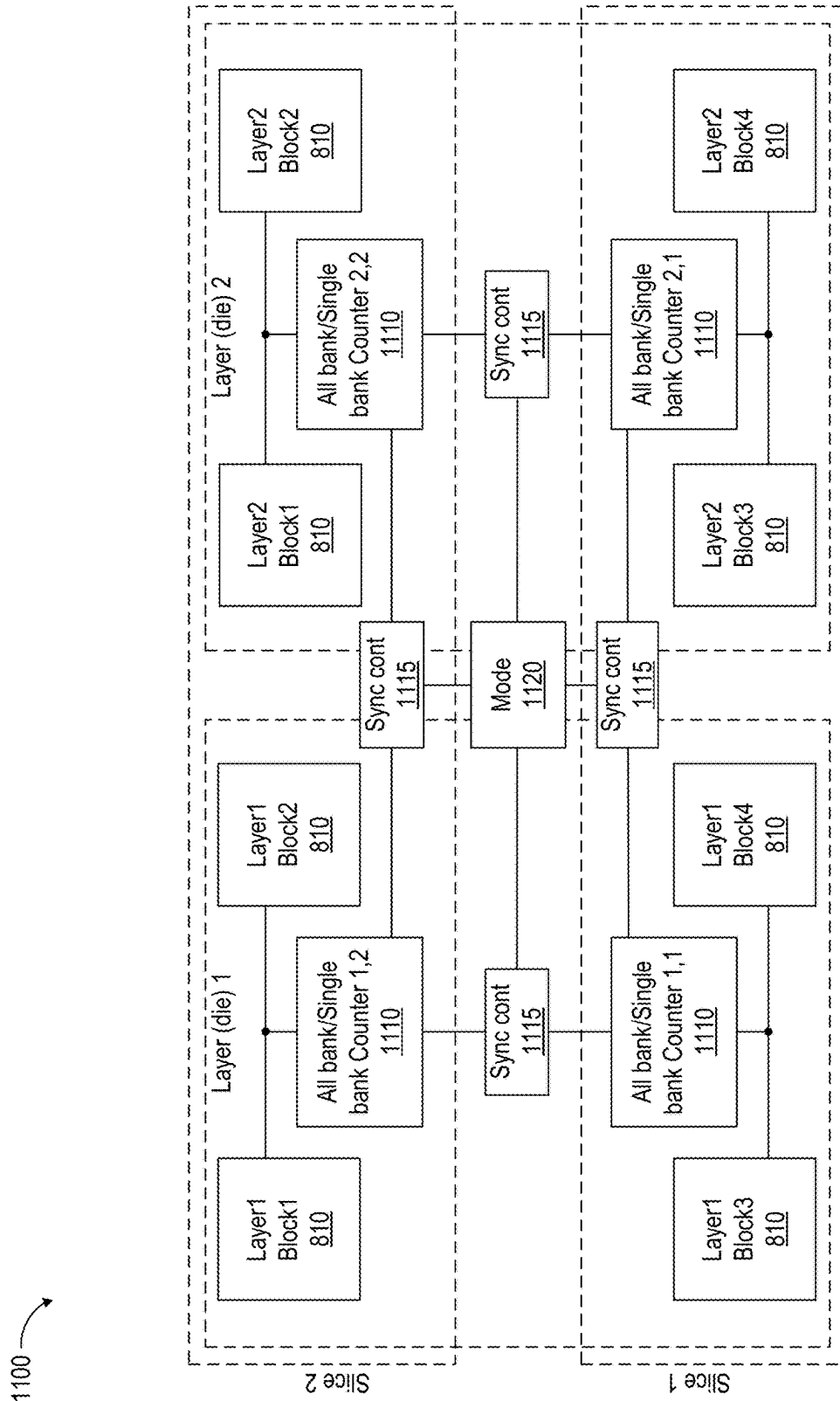


FIG. 11

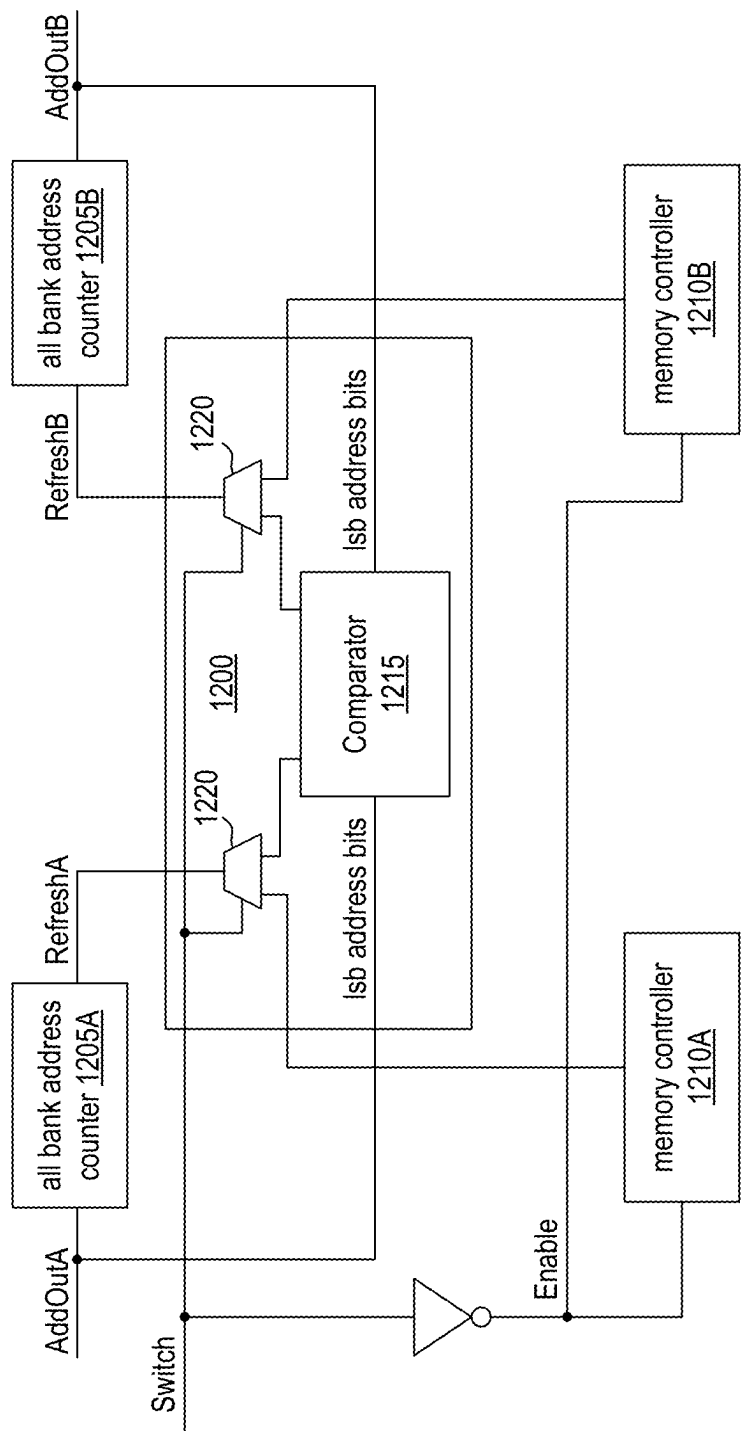


FIG. 12

1300

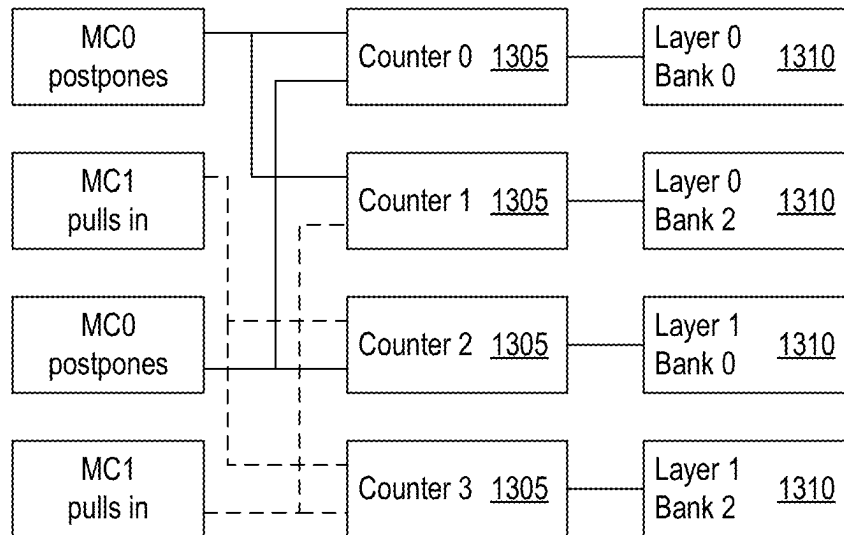


Fig. 13

1

MULTI-MODAL REFRESH OF DYNAMIC, RANDOM-ACCESS MEMORY

BACKGROUND

Dynamic, random-access memory (DRAM) includes storage cells that require their contents to be periodically refreshed. This is because information is held as charge across a capacitor, charge that leaks away over time. To prevent this leakage from destroying the information, the contents of each cell is periodically read and rewritten to restore the original amount of charge. Leaky buckets provide an apt analogy. Imagine storing a string of ones and zeros using a collection of leaky buckets, filling buckets to store a one and draining buckets to store a zero. If one were to wait too long, all the buckets would be empty and the stored ones lost. To preserve the ones, one might revisit each bucket from time to time to top off the partially filled buckets, and thus “refresh” the full value representing a one. The analogy weakens when one considers that modern DRAM devices have billions of such “buckets.” Managing refresh operations without losing data or unduly interfering with read and write operations is complicated, more so when refresh operations for a given quantity of DRAM are managed by multiple controllers with access to the same DRAM.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 illustrates a memory system 100 in which a host (e.g. a memory controller) 105 reads from and writes to DRAM memory in a three-dimensional (3D) stack 110.

FIG. 2 illustrates a memory system 200 in accordance with another embodiment.

FIG. 3 is a plan view of an embodiment of processing die 205 of FIG. 2 for implementing an artificial neural network.

FIG. 4 depicts ASIC 202 of FIG. 2, less base die 215, to illustrate how inter-die connections 400 formed using via fields 405 and related intra-die connectivity can afford processing units 220 fast, efficient access to underlying banks 225.

FIG. 5 depicts an embodiment of a processing unit 220 for a neural network.

FIG. 6 depicts a memory system 600 in accordance with an embodiment in which refresh operations for a stack of dram dies 120 can be directed by a host 105 or memory controllers MC 605 on a processing die 610.

FIG. 7 depicts a memory system 700 in accordance with another embodiment.

FIG. 8 depicts two refresh modes 800 and 805 for an HBM memory system with two DRAM dies (horizontal layers) and two DRAM slices (vertical stacks).

FIG. 9 depicts a memory system 900 that includes refresh control circuitry 905 for managing modes 800 and 805 of FIG. 8.

FIG. 10 depicts two additional refresh modes 1000 and 1005 using the same memory system illustrated in FIG. 8.

FIG. 11 depicts a memory system 1100 that includes refresh control circuitry for managing modes 1000 and 1005 of FIG. 10.

FIG. 12 depicts an example of sync control logic 1200 to show how control of a pair of refresh counters 1205A and 1205B is switched from a pair of memory controllers 1210A and 1210B to a third controller (not shown), such as an external host controller.

FIG. 13 is a block diagram 1300 illustrating refresh-counter synchronization in accordance with an embodiment

2

in which a refresh counter 1305 is provided for each of four independently accessible unit of memory 1310 (e.g. a bank or banks).

DETAILED DESCRIPTION

FIG. 1 illustrates a memory system 100 in which a host 105 with memory controller controllers MC1 and MC2 reads from and writes to DRAM memory in a three-dimensional (3D) stack 110 that includes a base die 115, DRAM dies 120, and a processing die 125. Processing die 125 includes address counters 130 and other elements that assist host 105 in managing refresh operations for arrays of memory banks 135 on DRAM dies 120. Processing die 125 also includes local memory controllers MC3 and MC4 that, in a mode selected using demultiplexers 145, can take over refresh management from all or a portion of memory banks 135, thus freeing host 105 for other tasks.

Each DRAM bank 135 is labeled to include a leading number indicative of the DRAM die and a trailing number indicative of the bank. “2B1” thus refers to the first DRAM bank on the second DRAM die. Each pair of banks 135 includes a request interface 150 to a row decoder 152 and a column decoder 155. Links 160 to each pair of banks communicate requests and data. Ignoring the data, requests on links 160 are conveyed along inter-die connections 165 from request interface 150 to one of demultiplexers 145 and to a request interface 150 on an adjacent DRAM die 120. Inter-die connections 165 from demultiplexers 145 to a vertical stack of memory-bank pairs (a “slice” of memory banks) can be made using e.g. through-silicon vias (TSVs) or Cu—Cu connections 167. Intra-die connections 170 on each DRAM die 120 likewise communicate requests and data in the plane of each DRAM die 120. Intra-die connections 170 on base die 115 connect to host 105 and, by way of vertical connections 175, to DRAM dies 120 and demultiplexers 145 on processing die 125.

Memory system 100 supports multiple modes of DRAM refresh, two in this example. In a first mode, host memory controllers MC1 and MC2 manage refresh operations for banks 135 on respective DRAM dies 120(1) and 120(2). Host 105 selects this mode by loading a register (not shown) with a mode value Mode of one, which causes demultiplexers 145 to present bank addresses from counters 130(1) and 130(2) to connections 170 on respective DRAM dies 120(1) and 120(2). Host 105 initiates refresh transactions by issuing refresh requests to stack 110. Refresh circuitry on processing die 125 includes refresh counters 130(1) and 130(2), each of which contains the address of a row to be refreshed in a bank of the corresponding DRAM die 120. Counters 130 can be instantiated on other layers. Refresh operations can follow various strategies, including “burst refresh” in which all rows are refreshed in a burst or “distributed refresh” in which rows are tracked such that refresh operations can be interspersed with read and write accesses. Whatever the strategy, this first mode essentially treats the collection of banks 135 on each memory die 120 as an independent memory.

In the second mode, local memory controllers MC3 and MC4 manage refresh operations for vertical slices of banks 135 on DRAM dies 120(1) and 120(2). Host 105 selects this mode by loading a register (not shown) with a mode value Mode of zero, which causes demultiplexers 145 to present bank addresses from counters 130(1) and 130(2) to connections 165 that extend to a subset—e.g. two of four—of banks 135 on each of DRAM dies 120(1) and 120(2). Controllers MC3 and MC4 issue refresh requests that initiate refresh

operations to row addresses specified by refresh counters **130(1)** and **130(2)**. Controllers **MC3** and **MC4** can employ the same or different refresh strategies as controllers **MC1** and **MC2**.

Processing die **125** is, in one embodiment, an accelerator die for a neural network that processes training data to derive machine-learning models. Host **105** can load DRAM dies **120** with training data, in the first mode, before placing stack **110** in the second mode to hand control over to processing die **125**. Processing die **125** can then execute a learning algorithm that relies on the training data to derive a function or functions optimized to achieve a desired result (e.g., to classify images). During this “training” phase, memory controllers **MC3** and **MC4** can manage refresh and other memory transactions for processing die **125**, eventually reporting the availability of derived model parameters or a time out to host **105**. Host **105** can then take back control, including of refresh transactions, and read out the model parameters from DRAM dies **120**. Learning algorithms can thus proceed with little or no interference from host **105**, which can similarly direct a number of neural networks in tandem.

Rather than await a report from stack **110**, host **105** can periodically read an error register (not shown) on stack **110** to monitor the progress of a learning algorithm. When the error or errors reaches a desired level, or fails to reduce further with time, processor host **105** can issue an instruction to stack **110** to return to the first mode and read out the optimized neural-network parameters-sometimes called a “machine-learning model”- and other data of interest.

In some embodiments stack **110** is only in one mode or the other. Other embodiments support more granular modality, allowing different banks to be directed by different external and internal memory controllers while avoiding bank conflicts. Embodiments that switch between modes to allow different controllers access to the same memory space support handoff protocols that ensure refresh operations are not postponed long enough to lose data between modes. Examples of protocols and supporting circuitry are detailed below.

FIG. 2 illustrates a memory system **200** in accordance with another embodiment. An ASIC **202** includes a processing die **205** bonded to a stack of DRAM dies **210** and a base die **215**. As used herein, the term “die” refers to an integrated-circuit die, a block of a semiconducting material, commonly silicon, upon and within which an integrated circuit is fabricated. In this example, processing die **205** incorporates an artificial neural network with an architecture that minimizes connection distances between processing units **220** and DRAM banks **225**, and thus improves efficiency and performance, while supporting externally and internally directed refresh operations. An exemplary processing unit **220** is detailed below in connection with later figures. This illustration outlines three refresh modes that allow a system-on-a-chip (SOC) **227** external to ASIC **202** and processing units **220** internal to ASIC **202** to take turns managing refresh operations for banks **225**.

Base die **215** includes a high-bandwidth memory (HBM) interface divided into four HBM sub-interfaces (not shown), each sub interface serving two of eight data channels Chan [7:0]. Using fields of TSVs **217** that extend through all intermediate dies, each data channel communicates with one of DRAM dies **210** and is supported by a corresponding request channel. SOC **227** can thus control read, write, and refresh operations independently for each DRAM die **210**. A refresh operation compatible with conventional HBM memory operations, but using refresh circuitry to be detailed

later, can be initiated by SOC **227** in the manner labeled **R1**, a bold arrow illustrating a refresh operation directed to a bank **225** in the uppermost DRAM die **210**. Though not shown, address counters and related support for refresh operations are integrated within one or more dies of ASIC **202**.

Processing die **205** includes eight channels Ch[7:0], one for each of corresponding HBM channels Chan[7:0], that allow requests and data to flow to and from processing units **220** using the same fields of TSVs **217** that afford access to SOC **227**. Each channel Ch[7:0] includes a pair of staging buffers **230**, a pair of memory controllers **235**, and at least one address counter **240**. Buffers **230** allow rate matching so that read and write data bursts from and to memory can be matched to regular, pipeline movement of an array of processing units **220**. In this context, a “processing unit” is an electronic circuit that performs arithmetic and logic operations using local, on-die memory or data provided from one or more of the memory dies. Processing units can operate as a systolic array, in which case they can be “chained” together to form larger systolic arrays. Memory controller **235**, including state machines or sequencers, can manage refresh operations and keep the processing pipeline running. Counter or counters **240** store addresses in support of refresh operations initiated by SOC **227**, memory controllers **235**, or by some other mechanism. A refresh operation initiated by one of memory controllers **235** is labeled **R2** with a neighboring bold arrow illustrating a refresh operation directed to a bank **225** in the uppermost DRAM die **210**.

Each processing units **220** additionally supports refresh operations in this embodiment. Each processing unit **220** includes an array of processing elements **242**, a sequencer **245**, and a TSV field **250** that connects to the data and request interfaces of each underlying DRAM bank **225**. Though not shown, each processing unit **220** has refresh circuitry, including one or more address counters, to manage refresh operations for the underlying column of banks **225**. In other embodiments, address counters and related overhead serve additional banks or collections of banks. A refresh operation initiated by one of processing units **220** is labeled **R3** with a neighboring bold arrow illustrating a refresh operation directed to one or more banks **225** in the underlying vertical “slice.” In other embodiments sequencer **245** can issue refresh instructions that make use of counts maintained in address counters **240**.

ASIC **202** can support any one or a combination of refresh modes simultaneously. For example, SOC **227** can write training data or read resolved models from a portion of the available DRAM banks **225** as processing die **205** refines the model or works on another model using another portion.

FIG. 3 is a plan view of an embodiment of processing die **205** of FIG. 2 for implementing an artificial neural network. Channels Ch[7:0] can be interconnected via one or more ring busses **305** for increased flexibility, for example to allow data from any channel to be sent to any tile, and to support use cases in which network parameters (e.g. weights and biases) are partitioned so that processing happens on portions of the neural network.

Processing units **220** can be described as “upstream” or “downstream” with respect to one another and with reference to signal flow in the direction of inference. Beginning with channel Ch6, the processing unit **220** labeled “I” (for “input”) receives input from one of staging buffers **230**. This input unit **220** is upstream from the next processing unit **220** toward the top. For inference, or “forward propagation,” information moves along the unbroken arrows through the chain of units **220**, emerging from the ultimate downstream

5

unit labeled “O” (for “output”) to another of staging buffers **230**. For training, or “back propagation,” information moves along the broken arrows from the ultimate downstream tile labeled “O,” emerging from the ultimate upstream tile labeled “I.”

Each processing unit **220** includes four ports, two each for forward propagation and back propagation. A key at the lower left of FIG. 3 shows shading that identifies in each unit **220** as a forward-propagation input port (FWDin), a forward-propagation output port (FWDout), a back-propagation input port (BPIn), or a back-propagation output port (BPout). Units **220** are oriented to minimize inter-unit connection distances. Processing elements **242** (FIG. 2) in each processing unit **220** can concurrently process and update partial results from both upstream and downstream processing elements and tiles in support of concurrent forward and back propagation.

FIG. 4 depicts ASIC **202** of FIG. 2, less base die **215**, to illustrate how inter-die connections **400** formed using via fields **405** and related intra-die connectivity can afford processing units **220** fast, efficient access to underlying banks **225**. As in the earlier examples, the layers are illustrated as separate but would be manufactured as stacked silicon wafers or dies interconnected using e.g. through-silicon vias (TSVs) or Cu—Cu connections so that the stack behaves as a single IC. The dies can be separate or in separate stacks in other embodiments. Banks **225** form a high-bandwidth memory with vertical slices for storing e.g. training data, partial results, and machine-learning models calculated during machine learning. Banks **225** can be complete banks or portions of banks (e.g. mats of bit cells). Each processing unit **220** can be equipped with a relatively simple memory controller—e.g. an address sequencer with refresh counter—that supports memory-access (read and write) and refresh operations. In other embodiments, each processing unit **220** can include a memory controller that manages refresh counters shared by more than one processing unit, such as controllers **235** of FIG. 2.

FIG. 5 depicts a processing unit **220** communicatively coupled to a via field **405**. A configurable switch **500** allows processing unit **220** to send data either to a downstream processing unit, as illustrated in FIG. 3, or to send requests and data to DRAM banks **225** using inter-die connections **400**, as illustrated in FIG. 4.

Processing unit **220** includes an array **242** of processing elements **510**. Processing unit **220** can be a “tile,” a geometric area on an IC die that encompasses a circuit that is or is largely replicated to form a tessellation of tiles. Switch **500** is depicted as outside of the tile for ease of illustration but switch **500** and the related connections can be integrated with other tile elements within the tile boundaries. Memory transactions that take place over via field **405** can be managed by sequencer **245** with access to a tile counter **515** or to a counter external to the tile.

Scratchpad and buffer logic **520** between the input and output nodes of array **242** can be included to store and buffer input and output signals. Sequencer **245** is of a simple and efficient class of memory controller that generates sequences of addresses to step through a microprogram, in this case to stream operands from and to memory banks **135** in underlying memory dies **210**. Sequencer **245** can also issue refresh instructions to addresses maintained in counter **515**.

FIG. 6 depicts a memory system **600** in accordance with an embodiment in which refresh operations for a stack of dram dies **120** can be directed by a host **105** or memory controllers MC **605** on a processing die **610**. Refresh logic **615** and refresh counters **620** similar to those introduced in

6

FIG. 1 are integrated with a base die **625** rather than on processing die **610**. This mode allows the stack to support refresh operations absent processing die **610**. DRAM dies **120** and base die **625** can thus function as high-bandwidth memory with or without the capability afforded by processing die **610**, and thus advantageously address a larger market. Connection points that communicate requests and addresses are highlighted. Dashed lines indicate information paths that traverse a die without a communicative coupling to intra-die components. Refresh operations can be supported in the manner detailed previously so a detailed discussion is omitted.

FIG. 7 depicts a memory system **700** in accordance with another embodiment. In this case, refresh operations for a stack of DRAM dies **705** can be directed by a host **105** or memory controllers MC **605** in a processor die **710** with the assistance of refresh logic **715** and refresh counters **720** integrated into each DRAM die **705**. Base die **725** is thus relatively simplified and may be omitted.

FIG. 8 depicts two refresh modes **800** and **805** for an HBM memory system with two DRAM dies (horizontal layers) and two DRAM slices (vertical stacks). Each layer includes four like-shaded blocks of memory **810**, each block representing one or more DRAM banks. Each layer of blocks can be accessed via one of two HBM pseudo channels **815** that are shaded to match the corresponding blocks of memory **810**. An external host (not shown) can independently access blocks in each layer from the corresponding pseudo channel **815** to perform read, write, and refresh transactions. A pair of vertical channels **820** are not used in mode **800** and are thus not shaded. Turning to mode **805**, pseudo channels **815** are not used and thus are not shaded; instead, integrated controllers (e.g. controller **235** of FIG. 2) can communicate via vertical channels **820**—slices—with like-shaded vertical collections of blocks to perform read, write, and refresh transactions.

The memory system can transition between modes **800** and **805** without losing state. An external host may write training data into DRAM via pseudo channels **815** in mode **800**, turn control over to internal controllers to develop and store model parameters in mode **805**, and take back control to read the model parameters in mode **800**. Control of refresh operations should transition between controllers without loss of data. Memory systems in accordance with some embodiments thus incorporate refresh-management circuitry that manages refresh addresses and timing while transitioning between refresh modes.

FIG. 9 depicts a memory system **900** that includes refresh control circuitry **905** for managing modes **800** and **805** of FIG. 8. Memory blocks **810** of FIG. 8 are labeled to illustrate their respective block and slice membership. Memory system **900** is functionally similar to system **100** of FIG. 1 but includes support for refresh-counter synchronization to manage transitions between modes **800** and **805** without loss of data. Refresh control circuitry **905** includes multiplexers **910(1)** and **910(2)** and respective refresh counters **915(1)** and **915(2)** that issue bank addresses for refresh operations to banks within blocks **810** of respective DRAM layers or slices, in dependence upon the selected one of modes **800** or **805**.

Whatever the mode, refresh control **905** allows the selected layers or slices to be managed independently. This independence improves performance because refresh operations directed to one subset of the DRAM (e.g., a layer or a slice) do not prevent the other subset from servicing memory requests. Different levels of refresh granularity can be used, but this embodiment supports per-bank refresh using coun-

ters **915(1,2)**. Each counter is actually two counters that support nested loops, one that the sequences through all bank addresses and the other that steps through row addresses within a selected bank. This and other refresh schemes are well known so a detailed discussion is omitted.

Refresh control **905** provides refresh scheduling flexibility that improves speed performance by allowing memory controllers to issue refresh commands early (pulled in) or late (postponed) to prioritize read and write memory requests. In one embodiment, for example, memory system **900** complies with the JEDEC DDR4 SDRAM Standard, which allows a memory controller to postpone or pull in up to eight all-bank refresh commands. Control is handed off between modes, however, with each counter serving a different set of memory banks. If counters **915(1)** and **915(2)** are too far out of synchronization when transitioning between modes, then the banks subject to the new controller are in danger of losing data. Otherwise a pulled-in address counter could issue addresses to a bank previously getting its addresses from a postponed address counter, thereby creating a hole in the address space even if the number of refreshes is correct. For example, the refresh addresses for the upper left memory block **810** (Layer 1, Block 1) are provided by counter **915(1)** in mode **800** and by counter **915(2)** in mode **805**. If the count applied by counter **915(2)** after a mode change is too far out from the count from counter **915(1)** then the data in layer 1, block 1, may be lost. Synchronization control **920** synchronizes counters **915(1,2)** to address this problem. In one embodiment, for example, when refresh control **920** receives a request to switch modes, it completes ongoing single-bank cycles and synchronizes the addresses of counters **915(1,2)** by stalling refresh requests for pulled-in counters and awaiting postponed counters to catch up. The internal or external memory controller assigned to the memory associated with each counter **915(1,2)** then takes control of memory access. In other embodiments, each collection of DRAM banks that remains together in the various modes is provided with its own counter.

FIG. 10 depicts two additional refresh modes **1000** and **1005** using the same memory system illustrated in FIG. 8. In mode **1000**, half of each layer of memory blocks **810** can be accessed using one of two HBM pseudo channels **815** that are shaded to match the corresponding blocks of memory **810**, and a slice of four memory blocks **810** can be accessed using one of the two vertical channels **820**. In mode **1005**, the bottom layer of memory blocks **810** can be accessed using one pseudo channel **815** and half of the memory blocks **810** on the top layer can be accessed using each of the two vertical channels **820**. Each of these modes allows memory access, including refresh control, to be managed by a combination of external and internal controllers.

Modes **1000** and **1005** can be implemented using a refresh scheme similar to what is conventionally termed “partial-array self-refresh” (PASR). PASR is an operational mode in which refresh operations are not performed across the entire memory but are instead limited to specific banks where data retention is required. Data outside of the active portion of the memory is not retained, and the resulting reduction in refresh operations saves power. For example, PASR may be used to refresh a subset of memory rows used to respond to baseband memory requests required to maintain connectivity to a local cellular network while other functionality is inactivated to preserve power. Methods and circuits in support of PASR are adapted in support of mixed-access modes of the type illustrated here. Considering mode **1000**, slice **2** is in service of one of vertical channels **820** but is

essentially “inactive” from the perspective of an external host employing pseudo channels **815** to access four of blocks **810** of the remaining slice. A memory system in mode **1000** could thus employ PASR-type methods and circuits to manage the memory available to an external host. Likewise, PASR-type methods and circuits can support internal memory controllers that have access to a subset of the memory blocks **810** along each vertical channel **820**.

FIG. 11 depicts a memory system **1100** that includes refresh control circuitry for managing modes **1000** and **1005** of FIG. 10. Each of modes **1000** and **1005** partitions memory such that up to three channels have concurrent access. Additional refresh counters **1110** and synchronization circuits **1115** are included to manage those counters during mode switching and responsive to a mode register **1120** that can be loaded by e.g. an external host. When mode register **1120** is loaded with a new mode value, each synchronization circuit **1115** completes ongoing single-bank cycles and synchronizes the addresses of counters **1110** by stalling refresh requests for pulled-in counters and awaiting postponed counters to catch up. The internal or external memory controller assigned to the memory associated with each counter **1110** then takes control of memory access.

FIG. 12 depicts an example of sync control logic **1200** to show how control of a pair of refresh counters, all-bank address counters **1205A** and **1205B**, is switched from a pair of memory controllers **1210A** and **1210B** to a third controller (not shown). Control logic **1200** includes an address comparator **1215** and two multiplexers **1220**. While memory controllers **1210A** and **1210B** are in control, multiplexers **1220** allow controllers **1210A** and **1210B** to issue refresh instructions and independently advance their respective counters **1205A** and **1205B** accordingly. Refresh registers **1205A** and **1205B** thus maintain separate addresses AddOutA and AddOutB for the memory under control of respective controllers **1210A** and **1210B**. Comparator **1215** monitors the least-significant bits (LSBs) of refresh addresses AddOutA and AddOutB to keep track of the difference between them. If the access mode is switched, illustrated here by the assertion of a signal Switch, independent controllers **1210A** and **1210B** are disabled and comparator **1215** issues refresh signals RefreshA or RefreshB of a number sufficient to catch up whichever of address counters **1205A** and **1205B** is behind.

FIG. 13 is a block diagram **1300** illustrating refresh-counter synchronization in accordance with an embodiment in which a refresh counter **1305** is provided for each of four independently accessible units of memory **1310** (e.g. a bank or banks). Memories **1305** can be accessed together in different refresh modes but each counter **1305** is permanently associated with the same memory **1310**. Refresh addresses thus do not require synchronization when transitioning between refresh modes.

Timing differences due to postponing or pulling in refresh transactions are settled before mode switching. Otherwise, postponed or pulled-in addresses could accumulate over time. Each memory controller MCO and MC1 keeps its status with respect to pulled-in or postponed refreshes. In that way, addresses will not be out-of-sync by more than four times the number of allowed pull-ins or postponements (one counter **1305** twice ahead, the other twice back). Some embodiments run refreshes at an increased rate. For example, upon mode switching the newly assigned memory controller can run refresh transactions twice through the available address space at twice the regular rate. Stopping the refresh counters at zero in the second round synchronizes all refresh counters without loss of data. In other embodi-

ments synchronization of the refresh addresses is accomplished by setting all addresses before mode switching to the value of the most postponed counter. These embodiments use additional logic to compare and set refresh addresses, and some rows would be refreshed more often than necessary, but no refreshes are required to catch up before switching between modes.

While the invention has been described with reference to specific embodiments thereof, it will be evident that various modifications and changes may be made thereto without departing from the broader spirit and scope of the invention. For example, additional stacked accelerator dies can be included with more or fewer DRAM dies, the accelerator die or a subset of the accelerator tiles can be replaced with or supplemented by one or more graphics-processing die or tiles, and the DRAM die or dies can be supplemented with different types of dynamic or non-volatile memory. Variations of these embodiments will be apparent to those of ordinary skill in the art upon reviewing this disclosure. Moreover, some components are shown directly connected to one another while others are shown connected via intermediate components. In each instance the method of interconnection, or “coupling,” establishes some desired electrical communication between two or more circuit nodes, or terminals. Such coupling may often be accomplished using a number of circuit configurations, as will be understood by those of skill in the art. Therefore, the spirit and scope of the appended claims should not be limited to the foregoing description. Only those claims specifically reciting “means for” or “step for” should be construed in the manner required under the sixth paragraph of 35 U.S.C. § 112.

What is claimed is:

1. An integrated circuit (IC) device comprising:

a. stacked first and second memory dies, each memory die having:

- i. a first memory-die request interface to a first memory bank;
- ii. a second memory-die request interface to a second memory bank; and
- iii. an intra-die connection to the first memory-die request interface and the second memory-die request interface;

b. a first inter-die connection between the first memory-die request interfaces of the first and second memory dies;

c. a second inter-die connection between the second memory-die request interfaces of the first and second memory dies; and

d. a multi-modal controller die coupled to the stacked first and second memory dies via the first and second inter-die connections, the multi-modal controller die to issue:

- i. in a first mode, first addresses to the intra-die connection of the first memory die and second addresses to the intra-die connection of the second memory die; and
- ii. in a second mode, first addresses to the first memory-die request interface in each of the first and second memory dies via the first inter-die connection and second addresses to the second memory-die request interface in each of the first and second memory dies via the second inter-die connection.

2. The IC device of claim 1, wherein the multi-modal controller die includes a first memory controller to issue the first and second addresses in the first and second modes.

3. The IC device of claim 1, wherein the multi-modal controller die includes a first counter to generate the first addresses and a second counter to generate the second addresses.

4. The IC device of claim 3, the multi-modal controller die further including a first memory controller connected to the first counter and a second memory controller connected to the second counter.

5. The IC device of claim 1, wherein the first and second memory banks comprise dynamic, random-access memory.

6. The IC device of claim 1, further comprising:

- e. a third inter-die connection to the intra-die connection of the first memory die; and
- f. a fourth inter-die connection to the intra-die connection of the second memory die.

7. The IC device of claim 6, the third inter-die connection to convey third memory addresses to the first and second memory-die request interfaces of the first memory die, and the fourth inter-die connection to convey fourth memory addresses to the first and second memory-die request interfaces of the second memory die.

8. The IC device of claim 6, further comprising a base die having third and fourth intra-die connections respectively coupled to the first and second intra-die connections.

9. The IC device of claim 1, the multi-modal controller die further including processing units communicatively coupled to the first and second memory dies to process data stored in the first and second memory banks of the first and second memory dies.

10. A method of accessing first and second stacked memory dies, each die having first and second memory banks, the method comprising:

issuing, in a first mode, first addresses to the first and second memory banks of the first memory die and second addresses to the first and second memory banks of the second memory die; and

issuing, in a second mode, the first addresses to the first memory banks of the first and second memory dies and the second addresses to the second memory banks of the first and second memory dies.

11. The method of claim 10, further comprising sending first and second refresh commands with the respective first and second addresses to the first and second memory banks in the first mode.

12. The method of claim 11, further comprising sending third and fourth refresh commands with the respective first and second addresses to the first and second memory dies in the second mode.

13. An integrated circuit (IC) device comprising:

a. stacked first and second memory dies, each memory die having first and second memory banks and an intra-die connection between the first and second memory banks;

b. a first inter-die connection between the first memory banks of the first and second memory dies;

c. a second inter-die connection between the second memory banks of the first and second memory dies; and

d. a third die coupled to the stacked first and second memory dies via the first and second inter-die connections, the third die to issue:

- i. first addresses to the first and second memory banks of the first memory die via the intra-die connection of the first memory die while issuing second addresses to the first and second memory banks of

11

the second memory die via the intra-die connection of the second memory die; and

- ii. third addresses to the first memory bank of each of the first and second memory dies via the first inter-die connection while issuing fourth addresses to the second memory bank of each of the first and second memory dies via the second inter-die connection.

14. The IC device of claim **13**, wherein the third die includes a memory controller to issue the first, second, third, and fourth addresses.

15. The IC device of claim **13**, wherein the third die includes a first counter to generate the first and third addresses and a second counter to generate the second and fourth addresses.

16. The IC device of claim **15**, the third die further including a first memory controller connected to the first counter and a second memory controller connected to the second counter.

12

17. The IC device of claim **13**, further comprising:

- e. a third inter-die connection to the intra-die connection of the first memory die; and
- f. a fourth inter-die connection to the intra-die connection of the second memory die.

18. The IC device of claim **17**, the third inter-die connection to convey fifth addresses to the first and second memory banks of the first memory die, and the fourth inter-die connection to convey sixth addresses to the first and second memory banks of the second memory die.

19. The IC device of claim **18**, further comprising a base die having third and fourth intra-die connections respectively coupled to the first and second intra-die connections.

20. The IC device of claim **13**, the third die further including processing units communicatively coupled to the first and second memory banks of the first and second memory dies to process data stored in the first and second memory banks of the first and second memory dies.

* * * * *